

Отчёт

Как Claude управляет QRKot через MCP

Устройство связки «модель → MCP-сервер → API» и разбор всех проблем, которые мы решили по пути

Проект	QRKot — благотворительный фонд поддержки котиков
Папка	C:\cat-charity-2-test
Репозиторий	github.com/spgritsun/cat-charity-2-test
Дата	3 октября 2026
Итог	Работает. Claude вызывает инструменты qrkot, вход под root@admin.ru проходит, список проектов читается из базы.

Что внутри

Часть	О чём
1. Как модель управляет API	Кто участвует, что такое инструмент, путь одного запроса от вашей фразы до записи в базе, вход по JWT, ошибки, безопасность.
2. Разбор проблем	Десять проблем этого дня: симптом, причина, как нашли, как решили и какой из этого урок.
3. Общие уроки	Приёмы диагностики, которые пригодятся в любом проекте, и шпаргалка команд.

Аналогия

Модель — гость ресторана, который знает только свой язык. Ваш API — кухня, которая принимает только заказы на своих бланках (POST /charity_project/, JSON, токен). MCP-сервер — официант-переводчик: у него есть меню с понятными описаниями блюд, он записывает заказ на правильный бланк, несёт на кухню и возвращает гостю результат. Почти весь этот отчёт — о том, как работает официант и что мешало ему дойти до кухни.

Часть 1. Как модель управляет API через MCP

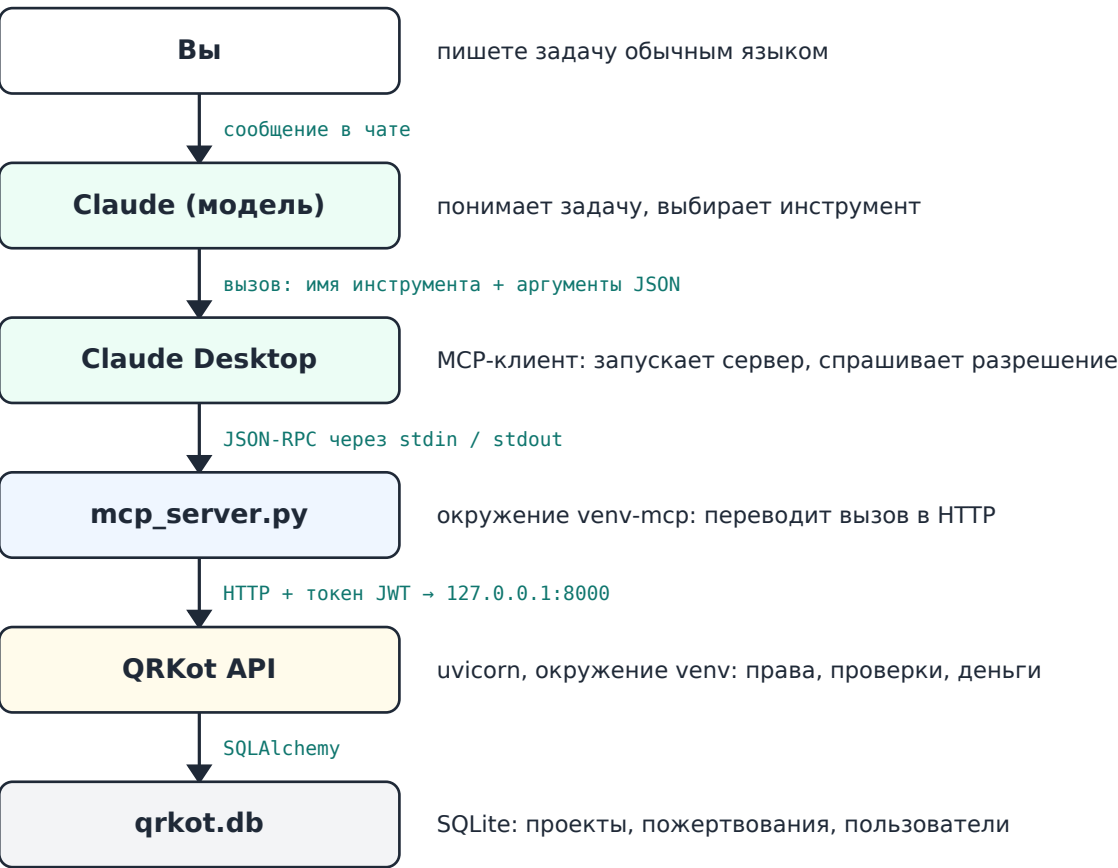
1.1. Главная мысль

Языковая модель сама ничего не делает в мире: она только пишет текст. Она не открывает соединения, не хранит токены и не знает адреса вашего сервера. Но модель умеет **просить** о действии в строгом формате: «вызови инструмент `create_project` с такими-то аргументами». Кто-то другой должен эту просьбу выполнить и вернуть результат.

MCP (Model Context Protocol) — стандарт, который описывает, как программы предлагают модели инструменты и как модель их вызывает. Он работает как розетка USB-C: один раз написали MCP-сервер, и его можно подключить к Claude Desktop, Claude Code, Cursor и другим клиентам без переделок.

1.2. Участники цепочки

Каждое звено умеет ровно одну вещь и общается только с соседями:



Ответ возвращается тем же путём снизу вверх: база → API → сервер → клиент → модель → вы.

Звено	Где живёт	Что знает и делает
Claude (модель)	серверы Anthropic	Видит список инструментов с описаниями. Решает, какой вызвать и с какими аргументами. Получает результат и пишет вам ответ.

Звено	Где живёт	Что знает и делает
Claude Desktop	ваш компьютер	MCP-клиент. При старте читает <code>claude_desktop_config.json</code> , запускает MCP-сервер как отдельную программу, передаёт ему вызовы и показывает вам запросы на разрешение.
mcp_server.py	ваш компьютер, окружение <code>venv-mcp</code>	Объявляет 9 инструментов. Входит в API по email и паролю, хранит JWT-токен, превращает вызов инструмента в HTTP-запрос, а ответ API — в результат или понятную ошибку.
QRKot API	ваш компьютер, <code>uvicorn</code> , окружение <code>venv</code>	Обычный FastAPI. Проверяет токен и права, валидирует данные, распределяет деньги и пишет в базу. О существовании модели не знает.

Урок

MCP-сервер — такой же клиент API, как Postman или Swagger, только управляет им модель. Поэтому модель может ровно то, что разрешено учётной записи из конфига, и ничего сверх этого. Если бы сервер ходил прямо в базу в обход API, все проверки (права, валидация, запрет удалять проект с деньгами) пришлось бы дублировать.

1.3. Что такое инструмент

Инструмент — это обычная асинх-функция Python с декоратором `@mcp.tool`. Библиотека FastMCP сама превращает её в описание для модели:

```
@mcp.tool(annotations=WRITE)
async def create_project(
    name: str, description: str, full_amount: int
) -> dict:
    """Создать новый проект (только суперпользователь).

    name — уникальное название, 5–100 символов;
    description — описание, не короче 10 символов;
    full_amount — сколько нужно собрать, целое число рублей больше 0.
    Свободные деньги из прошлых пожертвований сразу перейдут в проект.
    """
    return await api('POST', '/charity_project/', json={
        'name': name, 'description': description,
        'full_amount': full_amount,
    })
```

Часть функции	Во что превращается	Зачем модели
имя <code>create_project</code>	имя инструмента	по нему модель вызывает инструмент
<code>docstring</code>	описание инструмента	главное: по описанию модель решает, подходит ли инструмент, и узнаёт правила (длины, права, последствия)
аннотации типов	JSON-схема параметров	модель знает, что <code>full_amount</code> — целое число, и не пришлёт «пять тысяч»
<code>annotations=WRITE</code>	подсказки для клиента	клиент понимает, что вызов меняет данные, и спрашивает разрешение

Аналогия

Описание инструмента — это строчка в меню. «Блюдо №7» гостю ничего не скажет, а «Суп дня, острый, 15 минут» скажет. Модель выбирает инструменты именно по описаниям, поэтому хорошо написанный docstring важнее аккуратного кода внутри функции.

1.4. Девять инструментов QRKot

Инструмент	Запрос к API	Подсказка	Кому доступно
whoami	GET /users/me	только чтение	всем
list_projects	GET /charity_project/	только чтение	всем
create_project	POST /charity_project/	изменение	суперюзер
update_project	PATCH /charity_project/{id}	изменение	суперюзер
delete_project	DELETE /charity_project/{id}	разрушительное	суперюзер
donate	POST /donation/	изменение	авторизованным
my_donations	GET /donation/my	только чтение	авторизованным
all_donations	GET /donation/	только чтение	суперюзер
create_report	POST /yandex/	изменение	суперюзер

Подсказки «только чтение / изменение / разрушительное» передаются клиенту вместе с описанием. Что с ними делать, решает клиент и ваши настройки. Claude Desktop может спрашивать разрешение даже на читающие вызовы: так было сегодня с `all_donations`, который вы отклонили. Последнее слово всегда за вами.

1.5. Путь одного запроса: от фразы до базы

Разберём по шагам, что происходит, когда вы пишете: «Создай проект "Корм на зиму" на 5000 рублей».

№	Кто	Что происходит
1	Claude Desktop → модель	Вместе с вашим сообщением модели уходит список инструментов: имена, описания, схемы параметров.
2	Модель	Понимает задачу и подбирает инструмент <code>create_project</code> . Описания не хватает (нужно от 10 символов) — модель придумает его или спросит вас. Ответ модели — не текст, а вызов: <code>{"name": "Корм на зиму", "description": "...", "full_amount": 5000}</code> .
3	Claude Desktop	Видит подсказку «изменение» и показывает вам запрос на разрешение.
4	Claude Desktop → mcp_server.py	Передаёт вызов по протоколу JSON-RPC через stdin процесса.
5	mcp_server.py → API	Если токена ещё нет — <code>POST /auth/jwt/login</code> с email и паролем из конфига, токен запоминается. Затем <code>POST /charity_project/</code> с заголовком <code>Authorization: Bearer ...</code> .

№	Кто	Что происходит
6	QRKot API	Проверяет токен и права суперпользователя, валидирует поля, проверяет уникальность имени, создаёт проект, переводит в него свободные деньги и фиксирует всё одной транзакцией.
7	API → mcp_server.py → Claude Desktop	Ответ API (JSON проекта) возвращается как результат инструмента. Ошибка API — как ошибка инструмента с понятным текстом.
8	Модель → вы	Читает результат и пишет ответ обычным языком: «Проект создан, id 7, в него сразу перешло 0 рублей свободных средств».

1.6. Транспорт: как Claude Desktop общается с сервером

Для локальных серверов MCP использует транспорт **stdio**. Claude Desktop запускает команду из конфига как отдельный процесс и переписывается с ним через его стандартный ввод и вывод, как будто два человека передают друг другу записки под дверь. Сетевые порты для этого не нужны.

```
"mcpServers": {
  "qrkot": {
    "command": "C:\\cat-charity-2-test\\venv-mcp\\Scripts\\python.exe",
    "args":    ["C:\\cat-charity-2-test\\mcp_server.py"],
    "env": {
      "QRKOT_URL":      "http://127.0.0.1:8000",
      "QRKOT_EMAIL":    "root@admin.ru",
      "QRKOT_PASSWORD": "root"
    }
  }
}
```

- **command** — каким Python запускать. Именно поэтому здесь путь в venv-mcp: только там установлен FastMCP.
- **args** — какой файл выполнить.
- **env** — переменные окружения для процесса. Сервер читает их через `os.getenv`; в коде паролей нет.
- **stdout занят протоколом**, поэтому всё, что сервер пишет в `stderr`, Claude Desktop складывает в лог `%APPDATA%\Claude\logs\mcp-server-qrkot.log`.

Важно про перезапуск

Сервер запускается один раз при старте Claude Desktop и живёт, пока приложение открыто. Поэтому после любой правки `mcp_server.py` или конфига Claude Desktop нужно перезапустить **полностью**, включая значок в трее. Иначе работает старый код из памяти. Мы на это наступили в проблеме №10.

1.7. Вход и права: JWT

QRKot использует JWT — подписанный токен, который API выдаёт после входа по email и паролю. Дальше токен прикладывается к каждому запросу, как пропуск на проходной.

MCP-сервер делает так:

- При первом вызове любого инструмента входит в систему и запоминает токен в памяти процесса.
- Все следующие вызовы идут с этим токеном, поэтому повторный вход не нужен.
- Если API ответил 401 (токен истёк — по умолчанию через час), сервер один раз входит заново и повторяет запрос. Мы проверили это на токене со сроком жизни 2 секунды.
- Вход защищён блокировкой (`asyncio.Lock`), чтобы два одновременных вызова не входили дважды.

Урок

Права модели равны правам учётки из конфига. Для пожертвований хватит обычного пользователя; суперпользователь нужен, только если модель должна создавать, менять и удалять проекты. Пароль лежит в конфиге открытым текстом, поэтому используйте тестовую учётку, а не пароль от чего-то важного.

1.8. Ошибки — это обратная связь для модели

Сервер не падает при ошибке API, а превращает её в `ToolError` с человеческим текстом. Модель читает этот текст и может объяснить вам причину или исправить запрос сама. Все случаи ниже мы проверили вживую:

Ситуация	Что увидит модель
Имя проекта уже занято	Ошибка 400: Проект с таким именем уже существует!
Слишком короткое имя, отрицательная сумма	Ошибка 422: name: String should have at least 5 characters; full_amount: Input should be greater than 0
Правка закрытого проекта	Ошибка 400: Закрытый проект нельзя редактировать!
Обычный пользователь пытается удалить проект	Недостаточно прав (403): это действие доступно только суперпользователю.
Неверный пароль	Не удалось войти в QRKot под ... (Ошибка 400: LOGIN_BAD_CREDENTIALS)
uvicorn не запущен	API QRKot недоступен по адресу <code>http://127.0.0.1:8000</code> . Запущен ли uvicorn?
Нет токена Яндекс Диска	Ошибка 503: Яндекс Диск не настроен...

1.9. Почему у сервера своё окружение venv-mcp

FastMCP 4 требует pydantic 2.13 и свежий Starlette, а проект закреплён на pydantic 2.7.1 и Starlette 0.37.2 (так требует FastAPI 0.111). В одном окружении оба набора жить не могут: установка одного молча ломает другое.

Серверу сам проект не нужен: он общается с API по HTTP, как внешний клиент. Поэтому окружения разделены:

Окружение	Что внутри	Кто использует
venv	FastAPI 0.111, Starlette 0.37.2, pydantic 2.7.1, SQLAlchemy, XlsxWriter	uvicorn (API), тесты, PyCharm
venv-mcp	fastmcp 4.0.10, httpx 0.28.1, pydantic 2.13	только Claude Desktop для запуска mcp_server.py

Аналогия

Две кухни в одном здании, каждая со своим набором кастрюль. Повару API нужна старая кастрюля, повару MCP — новая. Пока кухни разные, все довольны. Стоит свалить посуду в один шкаф — и кто-то возьмёт не ту кастрюлю. Ровно это и случилось в проблеме №9.

Часть 2. Разбор проблем

Все проблемы этого дня в том порядке, в каком мы их встретили. У каждой — симптом (что вы видели), причина, как её нашли, решение и урок.

№	Проблема	Корень
1	Git считает изменёнными все файлы	переводы строк CRLF/LF
2	Git пишет «Another git process is running»	файл блокировки index.lock
3	Чистая установка падает без xlsxwriter	неполный requirements.txt
4	FastMCP конфликтует с проектом	разные версии pydantic
5	requirements-mcp.txt not found	команда в другой папке
6	pip из тестовой папки ставит пакеты в чужую	скопированный venv
7	47 Unversioned Files при коммите	копия проекта внутри проекта
8	MCP не может войти: пользователя нет	пустая учётка в базе
9	TypeError: unexpected keyword argument 'on_startup'	Sources Root в PyCharm
10	MCP не может войти, а браузер может	системный прокси Windows

Проблема 1. Git считает изменёнными все файлы

Симптом	Сразу после подключения папки <code>git status</code> показал M у всех файлов проекта, хотя вы ничего не правили.
Причина	Windows заканчивает строку двумя символами (CRLF, <code>\r\n</code>), Linux и репозиторий — одним (LF, <code>\n</code>). Git сравнивает байты, и каждая строка выглядит изменённой.
Как нашли	Команда <code>file app/main.py</code> показала «with CRLF line terminators», а <code>git diff --ignore-cr-at-eol</code> оставил ровно одно настоящее изменение — ваше логирование в <code>yandex_api.py</code> .
Решение	Один раз выполнить <code>git config core.autocrlf true</code> : git будет сам переводить концы строк при сохранении и чтении.

Урок

Если git показывает «изменено всё», сначала проверьте переводы строк, и только потом ищите, кто испортил файлы. Это как один и тот же текст с разными абзацными отступами: на вид разный, по сути одинаковый.

Проблема 2. Git пишет «Another git process seems to be running»

Симптом	Коммит или <code>git status</code> отказываются работать из-за файла <code>.git\index.lock</code> .
Причина	Это моя ошибка, причём дважды. Даже «только читающая» команда <code>git status</code> пытается обновить индекс и на время ставит табличку «занято» — <code>index.lock</code> . Убрать её за собой она не смогла: удалять файлы в вашей папке мне было запрещено.
Как нашли	Проверил время создания файла: оно совпало с моментом моей команды.
Решение	Первый раз вы удалили файл вручную. Второй раз я запросил разрешение на удаление, убрал файл сам и дальше запускал <code>git</code> с переменной <code>GIT_OPTIONAL_LOCKS=0</code> , при которой <code>status</code> индекс не трогает.

Урок

`index.lock` — пустая табличка, данных в ней нет. Если никакой `git` сейчас не работает, её можно спокойно удалить. А инструменты, которые лишь смотрят на репозиторий, должны запускать `git` без блокировок.

Проблема 3. Чистая установка падает без `xlsxwriter`

Симптом	На свежем окружении <code>pip install -r requirements.txt</code> проходит, а запуск приложения и тестов — нет: <code>ModuleNotFoundError: No module named 'xlsxwriter'</code> .
Причина	Модуль отчёта для Яндекс Диска импортирует <code>xlsxwriter</code> , а в <code>requirements.txt</code> его не было. У вас всё работало, потому что пакет стоял в окружении «с прошлого раза». Похожая история с <code>httpx</code> : он приезжал неявно, как зависимость <code>FastAPI</code> . А <code>FastMCP</code> 4 его вообще не тянет — первый запуск MCP-сервера упал именно на этом.
Как нашли	Установил зависимости в пустое окружение и запустил тесты. Чистая установка — лучший детектор забытых зависимостей.
Решение	Добавили <code>XlsxWriter==3.2.0</code> в <code>requirements.txt</code> , а <code>httpx</code> явно указали в <code>requirements-mcp.txt</code> .

Урок

Всё, что вы импортируете, должно быть в `requirements` явно, даже если сейчас оно «и так приезжает». Транзитивные зависимости — чужое обещание, которое могут отменить в любой версии.

Проблема 4. FastMCP конфликтует с проектом

Симптом	Предупреждение заранее, до установки: FastMCP требует pydantic 2.13, а проект закреплён на 2.7.1.
Причина	Две части системы требуют несовместимые версии одной библиотеки.
Как нашли	Поставил FastMCP в пустое окружение и посмотрел, какие версии он притянул.
Решение	Отдельное окружение <code>venv-mcp</code> и отдельный <code>requirements-mcp.txt</code> (подробно — в разделе 1.9).

Урок

Если двум программам нужны разные версии одной библиотеки и они общаются по сети, а не импортом, им нужны разные окружения. Это нормально, а не костыль.

Проблема 5. requirements-mcp.txt: No such file or directory

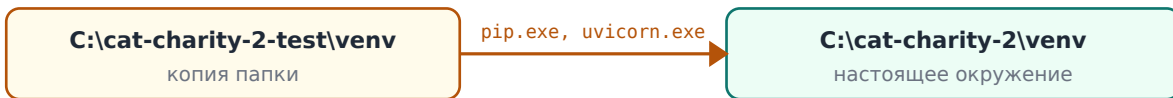
Симптом	<code>pip install -r requirements-mcp.txt</code> не находит файл.
Причина	Приглашение PowerShell было <code>PS C:\cat-charity-2></code> , а файлы лежали в <code>C:\cat-charity-2-test</code> . Относительный путь ищется в текущей папке.
Как нашли	Прочитали путь в начале строки PowerShell.
Решение	<code>cd C:\cat-charity-2-test</code> , затем повтор команды.

Урок

Перед командой с относительным путём смотрите на приглашение терминала: оно всегда показывает, где вы стоите. Особенно когда у двух папок почти одинаковые имена.

Проблема 6. pip из тестовой папки ставит пакеты в чужую

Симптом	Команда <code>venv\Scripts\pip install XlsxWriter</code> , запущенная в <code>C:\cat-charity-2-test</code> , ответила: «Requirement already satisfied ... in C:\cat-charity-2\venv ».
Причина	Папка <code>venv</code> была скопирована из другого проекта. Виртуальное окружение нельзя переносить: в <code>pip.exe</code> , <code>uvicorn.exe</code> и <code>activate</code> при создании зашивается абсолютный путь к своему Python.
Как нашли	Прочитал <code>venv\pyvenv.cfg</code> (строка <code>command = ... -m virtualenv C:\cat-charity-2\venv</code>) и заголовок внутри <code>pip.exe</code> .
Решение	Пока ничего не меняли: всё работает, но тестовая папка зависит от основной. Чтобы сделать её самостоятельной — пересоздать <code>venv</code> на месте и поставить зависимости заново.



Внутри `pip.exe` зашит путь:

`#!C:\cat-charity-2\venv\Scripts\python.exe`

Запуск «из тестовой папки» на деле идёт через библиотеки другого проекта.

Из-за этого я дважды ошибся в выводах. Сначала я решил, что в вашем окружении нет `xlswriter`: смотрел в библиотеки скопированной папки, которыми никто не пользуется. Потом лог скрипта показал, что настоящее окружение в порядке.

Урок

Окружение не копируют, а пересоздают: `python -m venv venv + pip install -r requirements.txt`. Скопированный `venv` — как ярлык на флешке: он открывает программу на старом диске.

Проблема 7. 47 Unversioned Files при коммите

Симптом	Окно коммита PyCharm показывает длинный список неотслеживаемых файлов.
Причина	46 из них — содержимое Новая папка/ (старая копия проекта внутри проекта), ещё один — пустой <code>fastapi.db</code> . Git честно спрашивает, следить ли за ними.
Как нашли	Сгруппировал неотслеживаемые файлы по верхней папке.
Решение	Не добавлять их в коммит. Копию удалить, перенести за пределы проекта или вписать в <code>.gitignore</code> .

Урок

Перед «Add all» всегда смотрите, что именно добавляется. Внутри Новая папка/ лежал `.env` с `SECRET`, паролем суперпользователя и токеном Яндекс Диска, а репозиторий публичный. Секрет, однажды попавший на GitHub, считается утёкшим, даже если коммит потом удалить.

Проблема 8. MCP не может войти: пользователя нет

Симптом	Первый вызов <code>whoami</code> через Claude Desktop: «Не удалось войти в QrKot под <code>root@admin.ru</code> ».
Причина	В вашей <code>qrkot.db</code> были только <code>user@example.com</code> , <code>sp.67@bk.ru</code> и <code>gritsun.sp@gmail.com</code> . Учётки <code>root@admin.ru</code> , прописанной в конфиге, не существовало.
Как нашли	Прочитал таблицу <code>user</code> напрямую из SQLite в режиме только для чтения.
Решение	Скрипт <code>create_root_user.bat</code> запустил имеющийся в проекте <code>setup_for_postman.py</code> , и тот создал суперпользователя <code>root@admin.ru / root</code> .

Урок

Ошибку входа проверяйте с двух сторон: верны ли данные и существует ли вообще такой пользователь в той базе, к которой подключён сервер. Здесь оказались правы обе догадки по очереди — см. проблему №10.

Проблема 9. `TypeError: unexpected keyword argument 'on_startup'`

Симптом	Запуск <code>app\main.py</code> из PyCharm падает на <code>router = APIRouter()</code> , и перед этим появляется <code>StarletteDeprecationWarning</code> .
Причина	В настройках PyCharm папка <code>venv-mcp\Lib\site-packages</code> была отмечена как Sources Root . PyCharm добавляет такие папки в <code>PYTHONPATH</code> раньше библиотек окружения. Python находил свежий Starlette из <code>venv-mcp</code> раньше Starlette 0.37.2 из <code>venv</code> , а FastAPI 0.111 со свежим Starlette несовместим: тот убрал параметр <code>on_startup</code> .
Как нашли	Ключевое наблюдение: скрипт <code>fix_venv.bat</code> тем же самым <code>python.exe</code> успешно импортировал приложение (<code>APP IMPORT OK</code>), а <code>pip</code> показывал правильные версии. Значит, пакеты в порядке, а отличается способ запуска. Ответ нашёлся в <code>.idea\cat-charity-2.iml</code> .
Решение	<code>File</code> → <code>Settings</code> → <code>Project Structure</code> : снять отметку <code>Sources</code> с <code>venv-mcp\Lib\site-packages</code> , а саму <code>venv-mcp</code> отметить как <code>Excluded</code> .

Обычный запуск (скрипт, консоль)

1. <code>C:\cat-charity-2-test\</code>	нет starlette
2. <code>venv\Lib\site-packages</code>	starlette 0.37.2
✓ Starlette 0.37.2 — работает	

Запуск из PyCharm (Sources Root)

1. <code>C:\cat-charity-2-test\</code>	нет starlette
2. <code>venv-mcp\Lib\site-packages</code>	новый starlette
3. <code>venv\Lib\site-packages</code>	до сюда не дошли

✗ **новый Starlette — TypeError**

Python просматривает папки сверху вниз и берёт первый найденный пакет.

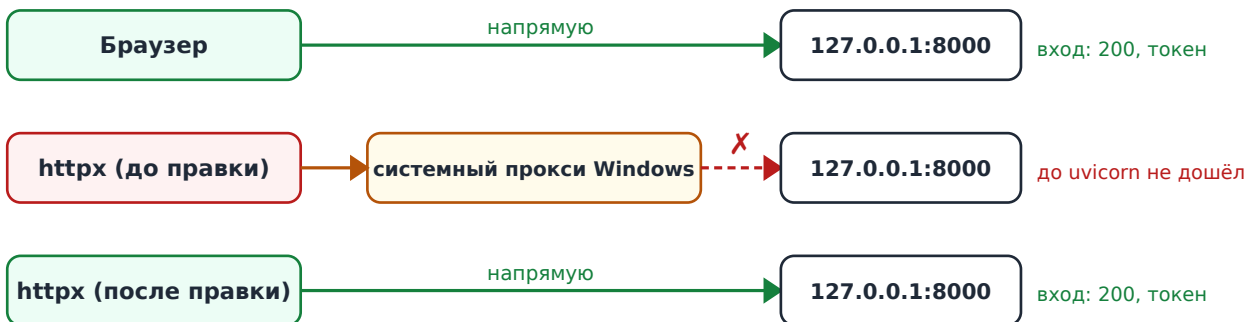
Python берёт первый найденный пакет с нужным именем. Как с двумя книжными шкафами: программа берёт книгу из того, что ближе к двери, а PyCharm поставил «новый» шкаф ближе.

Урок

Если программа работает из консоли и падает из IDE (или наоборот), ищите разницу в окружении запуска: `PYTHONPATH`, рабочую папку, переменные. Папку `site-packages` никогда не отмечают как `Sources Root`, даже если IDE предлагает это, чтобы убрать красное подчёркивание у импорта.

Проблема 10. MCP не может войти, а браузер может

Симптом	Пользователь <code>root@admin.ru</code> уже существует, <code>uvicorn</code> запущен, а <code>whoami</code> через MCP по-прежнему пишет «Не удалось войти».
Причина	<code>httpx</code> по умолчанию (<code>trust_env=True</code>) берёт настройки прокси из системы, на Windows — в том числе системный прокси, который включают VPN и прокси-программы. Запрос MCP-сервера к <code>127.0.0.1</code> уходил через прокси и не доходил до вашего <code>uvicorn</code> .
Как нашли	Сузили круг сравнением. (1) Во встроенном браузере открыл <code>/charity_project/</code> — API отвечает. (2) Тем же браузером отправил вход под <code>root@admin.ru / root</code> — 200 и токен. Значит, API, база и учётка в порядке, а отличается только клиент. (3) Отличие браузера от <code>httpx</code> в этой ситуации — отношение к прокси. Исправленный сервер проверил на своей стороне с намеренно прописанным нерабочим прокси: работает.
Решение	В <code>mcp_server.py</code> для локальных адресов отключили системный прокси, а в текст ошибки входа добавили код и причину от API, чтобы в следующий раз не гадать. После полного перезапуска Claude Desktop <code>whoami</code> вернул <code>root@admin.ru</code> , суперпользователь.



После правки `httpx` для локального адреса не смотрит в настройки прокси (`trust_env=False`).

```
USE_SYSTEM_PROXY = httpx.URL(API_URL).host not in (
    'localhost', '127.0.0.1', '::1')
...
async with httpx.AsyncClient(
    base_url=API_URL, timeout=REQUEST_TIMEOUT,
    trust_env=USE_SYSTEM_PROXY) as client:
```

Урок

Когда одно и то же действие работает одним способом и не работает другим, сравнивайте эти два способа, а не перепроверяйте общие части. И пишите сообщения об ошибках так, чтобы в них была причина: расплывчатое «не удалось войти» стоило нам лишнего круга.

Что осталось сделать

Задача	Зачем
Закоммитить правку <code>mcp_server.py</code>	обход прокси и подробная ошибка входа пока есть только у вас на диске
Удалить <code>fix_venv.bat</code> , <code>fix_venv.log</code> , <code>create_root_user.bat</code>	они были нужны только для починки
Решить судьбу Новая папка/ и <code>fastapi.db</code>	убрать риск случайно опубликовать <code>.env</code>
Пересоздать <code>venv</code> в тестовой папке (по желанию)	чтобы она не зависела от <code>C:\cat-charity-2</code>
Исправить проверку имени при редактировании проекта	сейчас нельзя сохранить проект с его же названием (400). Модель наткнётся на это, если пришлёт имя вместе с новым описанием

Часть 3. Общие уроки

3.1. Как искать причину, когда «не работает»

- **Разбейте цепочку на звенья и проверьте каждое отдельно.** Claude Desktop → MCP-сервер → API → вход → база. Мы видели, что инструменты `qrkot` появились (сервер запущен), API ответил (`uvicorn` жив), и сузили поиск до входа.
- **Сравнивайте работающий и неработающий случаи.** Скрипт против PyCharm (проблема 9), браузер против `httpx` (проблема 10). Причина почти всегда в том, чем они различаются.
- **Проверяйте факты, а не предположения.** Версии смотрите через `pip show`, пользователей — в самой базе, пути — в `ruvenv.cfg`. Мои ошибочные выводы в проблеме 6 появились именно там, где я предположил, а не проверил.
- **Читайте первую строку предупреждения.** `StarletteDeprecationWarning` сразу говорил, что `Starlette` не той версии, — ещё до трейсбека.
- **Делайте ошибки информативными.** Хорошее сообщение экономит целый круг диагностики.

3.2. Правила, которые сэкономят время

Правило	Какую проблему предотвращает
Одно окружение на одну задачу; окружения не копируют, а пересоздают	4, 6, 9
Всё, что импортируется, — явно в <code>requirements</code>	3
Проверять установку на чистом окружении	3
<code>core.autocrlf true</code> на Windows	1
Смотреть на приглашение терминала перед командой	5
Не добавлять в <code>git to</code> , что не просматривали; <code>.env</code> только в <code>.gitignore</code>	7
Не отмечать <code>site-packages</code> как <code>Sources Root</code>	9
Для локальных адресов не использовать системный прокси	10
После правок MCP-сервера полностью перезапускать Claude Desktop	10

3.3. Шпаргалка команд

```
:: запустить API (из папки проекта)
cd C:\cat-charity-2-test
venv\Scripts\uvicorn app.main:app

:: проверить версии в окружении
venv\Scripts\pip show starlette fastapi pydantic

:: проверить совместимость установленных пакетов
venv\Scripts\pip check

:: установить зависимости MCP-сервера (только в venv-mcp!)
venv-mcp\Scripts\pip install -r requirements-mcp.txt

:: переводы строк на Windows
git config core.autocrlf true

:: лог MCP-сервера в Claude Desktop
type "%APPDATA%\Claude\logs\mcp-server-qrkot.log"
```

3.4. Файлы MCP-интеграции

Файл	Назначение
mcp_server.py	9 инструментов, вход по JWT, обработка ошибок, обход прокси для локального API
requirements-mcp.txt	fastmcp 4.0.10 и httpx 0.28.1 для окружения venv-mcp
MCP_README.md	установка, подключение к Claude Desktop и Claude Code, безопасность, диагностика
%APPDATA%\Claude\claude_desktop_config.json	блок mcpServers → qrkot: чем и с какими переменными запускать сервер